

Chapter 10 Class and Objects

Structure in C: -

A structure can be used to create **user defined datatypes** but all the members of structure are public by default. In c we can't declare members of structure as private so we can't hide data from external objects. "struct" is a keyword in C and also in C++.

Example: - This example shows the use & syntax of structure.

```
#include<iostream>
struct A{
    int x;           //members are public by default
    int y;
};
void getdata(A &r){
    cout<<"Enter two numbers:";
    cin>>r.x>>r.y;
}
void display(A obj){
    cout<<obj.x<<"\t"<<obj.y<<endl;
}
int main(){
    A a1,a2;
    //a1.x=5;           //object can directly access its members.
    getdata(a1);         //object is not calling function.
    getdata(a2);         //object is passed as an argument.
    cout<<"Values are"<<endl;
    display(a1);
    display(a2);
    return 0;
}
```

Note: -

In the above program data is not secure because any object can directly access its members so members should be declared as private but this option is not available in structure of c so class chapter is introduced in C++ to provide data security. Also the functions are defined outside the structure so they are not the members of structure.

Class: -

Class is also used to create a **user defined datatype**. It provides both public and private modifiers. "class" is a keyword. A class is a collection of member variables (called as data) and member functions (called as operations). A class is a way to bind the data and its associated functions together. It allows the data and functions to be hidden, if necessary, from external objects. When defining a class, we are creating a new abstract data type that can be treated like any other built-in data type.

Data members are the data variables and member functions are the functions used to manipulate these variables and together these data members and member functions defines the properties and behavior of the objects in a Class.

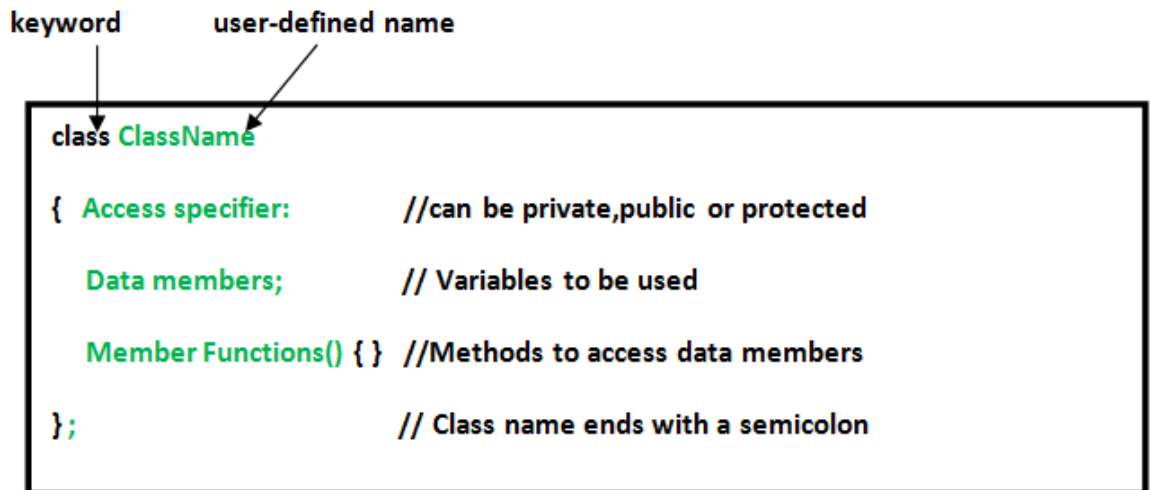
Matrix Computers (Page 2 of 12)

10/564 Kaveri Path, Mansarovar Jaipur-302020 Rajasthan
9414752750, 9414930104, 9414244351, 0141-2399633

Object: -

An **Object** is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated.

When a class is defined, only the specification for the object is defined; no memory or storage is allocated. To use the data and access functions defined in the class, we need to create objects.



Static and Dynamic Object: -

Object created without new keyword is called as static object. There is no need to explicitly delete static object it is automatically deleted when goes out of scope. We should not return any local static object from a function.

Object created using new keyword is called as dynamic object. We have to delete a dynamic object using delete keyword. It will not be deleted even if it goes out of scope. We can return dynamic object from a function.

```
class A{  
  
};  
int main(){  
    A a1;           //static object  
    A *p=new A;     //Dynamic object  
    delete p;  
    return 0;  
}
```

Accessing data members and member functions: -

The data members and member functions of class can be accessed using the dot('.') operator with the object. For example if the name of object is *obj* and you want to access the member function with the name *printName()* then you will have to write *obj.printName()* .

Access Modifiers: -

Access modifiers are used to implement an important aspect of Object-Oriented Programming known as **Data Hiding**.

There are 3 types of access modifiers available in C++ and these are keywords also:

1. **public**
2. **private**
3. **protected**

Note: If we do not specify any access modifiers for the members inside the class then by default the access modifier for the members will be **Private**.

1. public:

All the class members declared under the public specifier will be available to everyone. The data members and member functions declared as public can be accessed by other classes and functions too. The public members of a class can be accessed from anywhere in the program using the direct member access operator (.) with the object of that class.

2. private:

The class members declared as *private* can be accessed only by the other member functions. They are not allowed to be accessed directly by any object or non-member function. Only the member functions or the friend functions are allowed to access the private data members of a class.

3. protected:

This will be covered in inheritance later.

Example: - This example shows the use & syntax of class.

```
#include<iostream>
class A{
    private:
        int x;
        int y;
    public:
        void getdata(){
            cout<<"Enter two numbers:";
            cin>>x>>y;
        }
        void display(){
            cout<<x<<"\t"<<y<<endl;
        }
};
int main(){
    A a1,a2;
    //a1.x=5;    object can't access its member directly so error
    a1.getdata();
    a2.getdata();
    a1.display();
```

Matrix Computers (Page 4 of 12)

10/564 Kaveri Path, Mansarovar Jaipur-302020 Rajasthan
9414752750, 9414930104, 9414244351, 0141-2399633

```
a2.display();  
return 0;  
}
```

Note:-

In the above program no object can access the data directly so data is secured. All data members are private so object can access them through public functions only. We can apply conditions in function to check whether the data input is correct or not. All the functions are defined inside the class so they are the members. All the functions are called through object because they are the members of the class.

We Provide online/offline training in

C	C++	DSA	JAVA	Adv. Java
Web. Designing	PHP	Python	Django	ML
RSCIT	RSCFA	Office Automation	Tally ERP & Prime	
Advance Excel	DTP	Photoshop	Corel Draw	

Special classes to solve questions of hackerrank.com, hackerearth.com, codechef.com

Free Demo Classes	Repeat Option Available	Free Study Material
-------------------	-------------------------	---------------------

We provide internship, summer training for B. Tech students

Structure in C++: -

In C++ Structure is improved. Now we can use public and private modifiers also. But if we do not specify any modifier then it will assume public by default. We can define functions as members of structure also. But programmers prefer to use class not structure now.

Inline v/s functions defined outside the class: -

All the functions defined inside the class are inline by default. It means compiler will add inline keyword automatically before each and every function defined inside the class. But if a function is too complex or long and can't be treated as inline then

Matrix Computers (Page 5 of 12)

10/564 Kaveri Path, Mansarovar Jaipur-302020 Rajasthan
9414752750, 9414930104, 9414244351, 0141-2399633

compiler will give a warning. To remove this warning we have to define the function outside the class.

A member function defined inside the class automatically become inline but if a member function is defined outside the class then we can decide whether we want to define it inline or not using inline keyword. So it is better to define all members outside the class.

If we want to define a member function outside the class then we have to declare it inside the class also and we have to prefix the name of the function with class name and scope resolution operator in the definition.

Example: - This example is same as previous example but member functions are defined outside the class so they are not an inline function.

```
#include<iostream>
class A{
    int x,y;
    public:
        void getdata();
        void display();
};
void A::getdata(){
    cout<<"Enter two numbers:";
    cin>>x>>y;
}
void A::display(){
    cout<<x<<"\t"<<y<<endl;
}
int main(){
    A a1;
    a1.getdata();
    a1.display();
    return 0;
}
```

Generally, a class specification has two parts:

1. **Class declaration: -**
It describes the type and scope of its members.

```
class class_name{
    private:
        variable declarations;
        function declarations;
    public:
        variable declarations;
        function declaration;
};
```

2. **Class function definitions: -**

Matrix Computers (Page 6 of 12)

10/564 Kaveri Path, Mansarovar Jaipur-302020 Rajasthan
9414752750, 9414930104, 9414244351, 0141-2399633

It describes how the class functions are implemented. Member functions that are declared inside a class have to be defined separately outside the class. To define a member function outside the class we must prefix class name & scope resolution operator before function name. If we prefix the keyword "inline" then the function defined outside the class will be an inline function.

```
return-type  class-name :: function-name(argument declaration)
{
    function body
}
```

Characteristics of a member function: -

- Several different classes can use the same function name.
- Member functions can access the private data of the class. A non-member function cannot do so.
- A member function can call another member function directly, without using the dot operator.

"this" keyword: -

"this" is a pointer inside the class which stores the address of the function calling object. Every class contains only one this pointer irrespective of number of objects. When we call a member function then the address of the calling object is passed as an argument to the function and this address will be received by "this" pointer inside the function definition. The name of every class member variables will be prefixed by "this->" automatically. This is called as implicit use of "this".

Sometimes the name of a local variable is same as of member variable then priority will be given to local variable. It means local variable will override instance member variable. This is called as instance variable hiding. Now to use member variable in this case we have to prefix "this->" before member name. This is called as explicit use of "this".

Example: -

```
class A{
    int x;                                //member
    void f1(){
        x=5;
        int x=10;                        //local
        cout<<x<<endl;                    //10
        cout<<this->x<<endl;                //5
    }
}
```

Nesting of member functions: -

When we call a member function within another member function then it is called as nesting.

Example: -

```
#include<iostream>
using namespace std;
```

Matrix Computers (Page 7 of 12)

10/564 Kaveri Path, Mansarovar Jaipur-302020 Rajasthan
9414752750, 9414930104, 9414244351, 0141-2399633

```
class A{
    public:
    void f1(){
        cout<<"f1 called"<<endl;
    }
    void f2(){
        f1();
        cout<<"f2 called"<<endl;
    }
    void f3(){
        f2();
        cout<<"f3 called"<<endl;
    }
};
int main(){
    A a1;
    a1.f3();
    return 0;
}
```

Single Ton class: -

It is a class in which we can create only one object. Multiple objects can't be created. This class is useful when we don't want to share resources.

Overloading of member functions: -

We can overload member functions just like global functions.

Example: -

```
#include<iostream>
using namespace std;
class MyMath{
    public:
    int sum(int a,int b){
        return(a+b);
    }
    int sum(int a,int b,int c){
        return(a+b+c);
    }
};
int main(){
    A a1;
    cout<<a1.sum(5,6)<<endl;
    cout<<a1.sum(5,6,7)<<endl;
    return 0;
}
```

Chained function calls: -

Matrix Computers (Page 8 of 12)

10/564 Kaveri Path, Mansarovar Jaipur-302020 Rajasthan
9414752750, 9414930104, 9414244351, 0141-2399633

If we want to call multiple functions of a class in a single statement, then we can use chained function call. But to implement this we have to return the calling object from the function using *this.

Example: -

```
#include<iostream>
using namespace std;
class A{
public:
    A f1(){
        cout<<"f1 called"<<endl;
        return *this;
    }
    A f2(){
        cout<<"f2 called"<<endl;
        return *this;
    }
    A f3(){
        cout<<"f3 called"<<endl;
        return *this;
    }
};
int main(){
    A a1;
    a1.f1().f2().f3();
    return 0;
}
```

Nested Class: -

A nested class is a class which is declared in another enclosing class. A nested class is a member and as such has the same access rights as any other member. The members of an enclosing class have no special access to members of a nested class; the usual access rules shall be obeyed.

Example: -

```
#include<iostream>
using namespace std;
class A{           //Outer class
public:
    int x;
    void f1(){
        cout<<"Value of x from outer class "<<x<<endl;
        B b1;
        //b1.y=6;      error
        //cout<<"Value of y from outer class" <<b1.y<<endl; error
    }
    class B{       //Inner class or nested class
    public:
```


Matrix Computers (Page 9 of 12)

10/564 Kaveri Path, Mansarovar Jaipur-302020 Rajasthan
9414752750, 9414930104, 9414244351, 0141-2399633

```
int y;
void f2(){
    A a1;
    a1.x=5;
    cout<<"Value of x from inner class "<<a1.x<<endl;
    cout<<"Value of y from inner class "<<y<<endl;
}
};
};
int main(void){
    A a1;
    a1.x=5;
    a1.f1();
    A::B b1;
    b1.y=6;
    b1.f2();
    return 0;
}
```

Local class: -

A class declared inside a function becomes local to that function and is called Local Class. We can declare object of local class inside that function only.

Example: -

```
#include<iostream>
using namespace std;
class A{
public:
    void f1(){
        class B{
        public:
            int y;
            void f2(){
                cout<<"inner class "<<endl;
            }
        };
        B b1;
        b1.y=6;
        b1.f2();
        cout<<"Value of y "<< b1.y<<endl;
    }
};
int main(void){
    A a1;
    a1.f1();
    return 0;
}
```

Type Conversion in user defined datatypes: -

We can't convert one user defined datatype into another user defined datatype using implicit or explicit typecast. To allow this we have to overload typecast operator which will be covered later.

```
class A{
    public:
        int x;
};
class B{
    public:
        int y;
};
int main(){
    A a1;
    A1.x=5;
    B b1;
    b1=a1;        //error
    b1=(B)a1;     //error
    return 0;
}
```

Has a relation/Containership/Composition: -

We can create an object of one class into another and that object will be a member of the class. This type of relationship between classes is known as containership or has_a relationship as one class contain the object of another class. And the class which contains the object and members of another class in this kind of relationship is called a container class.

The object that is part of another object is called contained object, whereas object that contains another object as its part or attribute is called container object.

```
class Date{
    public:
        int day,month,year;
}
class Student{
    int roll;
    char name[20];
    Date dob;
}
```

Object-oriented programming: -

As the name suggests uses objects in programming. Object-oriented programming aims to implement real-world entities like inheritance, hiding, polymorphism, etc in programming. The main aim of OOP is to bind together the data and the functions that operate on them so that no other part of the code can access this data except that function.

Features of OOPS:-

1. **Object:** object is a real world entity. Entity is something which has existence in real world. An Object is an identifiable entity with some characteristics and behaviour. An Object is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated.
2. **Class:** class is a collection of similar type of objects. Class is defined to store common properties of similar type of objects. It is a user-defined data type, which holds its own data members and member functions, which can be accessed and used by creating an instance of that class. A class is like a blueprint for an object.
3. **Data Hiding:** By declaring the member variables as private we can hide them from external objects.(For Data Security)
4. **Data Encapsulation:** Bundling of member variables and member functions together in a class as a single unit. In normal terms, Encapsulation is defined as wrapping up of data and information under a single unit. In Object-Oriented Programming, Encapsulation is defined as binding together the data and the functions that manipulate them.
5. **Data Abstraction:** To hide the internal complex details of a class from external objects to make their work simple. Data abstraction is one of the most essential and important features of object-oriented programming in C++. Abstraction means displaying only essential information and hiding the details. Data abstraction refers to providing only essential information about the data to the outside world, hiding the background details or implementation.
6. **Inheritance:** The capability of a class to derive properties and characteristics from another class is called Inheritance. Inheritance is one of the most important features of Object-Oriented Programming.
7. **Polymorphism:**
 - a. function overloading: Function overloading is using a single function name to perform different types of tasks.
 - b. operator overloading: The process of making an operator to exhibit different behaviours in different instances is known as operator overloading.
8. **Dynamic Binding:** In dynamic binding, the code to be executed in response to function call is decided at runtime. C++ has virtual functions to support this.
9. **Message Passing:** When we call a member function through object it is called as message passing. Objects communicate with one another by sending and receiving information to each other. A message for an object is a request for execution of a procedure and therefore will invoke a function in the receiving

object that generates the desired results. Message passing involves specifying the name of the object, the name of the function and the information to be sent.

MATRIX